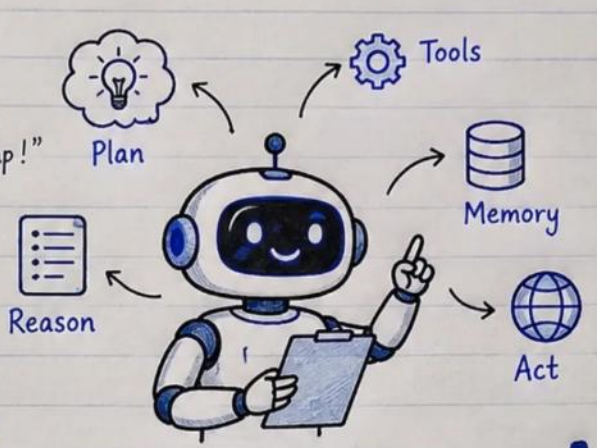
"AI that doesn't just respond, but takes action is the next big leap!"



→ Perceive
→ Reason
→ Plan
→ Use Tools
→ Take Action
→ Learn
→ Achieve Goals

Agentic AI

Complete Handwritten Study Handbook

15 Pages

"From answering questions to taking real actions in the neal world."

Key Areas Covered :

- Concepts & Foundations
- Architecture & Components
- Memory & Tools
- Planning & Reasoning
- RAG in Agentic AI
- Multi-Agent Systems
- Frameworks
- Real-world Applications
- Challenges & Safety
- Interview Questions & Revision

Real-world Applications :

AI Assistants | Research Agents | Coding Agents | Automation Agents | Business Agents

"Autonomous Agents for a smarter, more productive future."

"Insta ID: @aditechnova18

SUBTOPIC: Introduction to Agentic AI

1) What is Agentic AI ?

Agentic AI refers to AI systems that can autonomously plan, make decisions, use tools, take actions and learn from the environment to achieve specific goals with minimal human intervention.

> "It's not just about generating answers, but about taking actions towards achieving a goal."

2) Key Characteristics

- Autonomy – can operate independently
- Goal-oriented – works towards specific goals
- Reasoning – can analyze, plan and decide
- Tool Use – can use external tools/APIs
- Memory – can remember past interactions
- Adaptability – can learn and improve
- Action Execution – takes real-world actions
- Continuous Interaction – interacts with environment and users

3) Evolution of AI

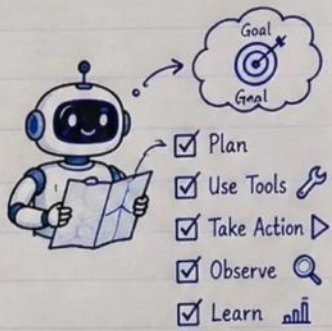
Traditional AI
- Rule-based
- Predefined logic
- Handles specific tasks
- No learning

→

Generative AI (LLMs)
- Generates text, images, code etc.
- Understands and generates content
- Mostly reactive
- Limited real-world actions

→

Agentic AI
- Plans and reasons
- Uses tools
- Takes actions
- Learns from experience
- Works towards goals autonomously



4) Comparison Table

Feature	Traditional AI	Generative AI	Agentic AI
Main Purpose	Solve specific tasks	Generate content	Achieve goals
Approach	Rule-based / ML models	Deep learning (LLMs)	Planning + Reasoning + Action + Learning
Autonomy	Low	Low (mostly reactive)	High
Tool Use	Limited	Limited	Extensive
Memory	Minimal	Limited context	Short-term + Long-term
Output	Classification / Prediction	Text, images, code etc.	Actions + Results
Real-world Interaction	No	Indirect	Yes (via tools/APIs)
Example	Spam filter	ChatGPT	AI Research Agent, Coding Agent, Automation Agent

5) Real-world Examples of Agentic AI

- AI research assistant – searches, reads papers, summarizes and writes report
- Coding agent – writes, tests and debugs code
- Task automation agent – manages emails, schedules, files etc.
- Travel planning agent – books tickets, hotels, creates itinerary
- Business agent – analyzes data and takes business actions
- Personal assistant agent – handles daily tasks using tools

(6) Why is Agentic AI Important?

- Solves complex multi-step problems
- Increases productivity and automation
- Can work across multiple tools and systems
- Enables real-world impact
- Foundation for future AI-driven applications

Insta ID: @aditechnova18

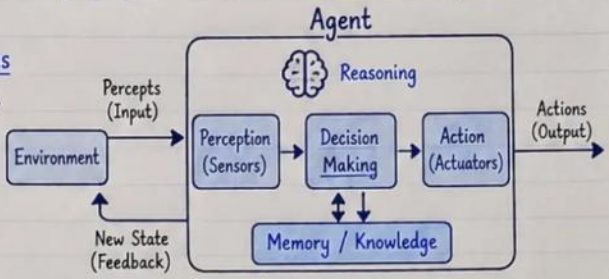
SUBTOPIC: AI Agents and Their Types

1) What is an AI Agent ?

An AI agent is an intelligent system that perceives its environment, makes decisions and takes actions to achieve specific goals.

> "An AI agent is an entity that can sense, reason, act and learn to achieve goals autonomously."

2) General Structure of an AI Agent



3) Types of AI Agents

1) Reactive Agents

- Simple agents that respond to current percepts only.
- No memory of past states.
- Work on condition-action rules.

Example: Automatic doors, basic chatbots.



2) Goal-based Agents

- Use goals to decide actions.
- Consider future states and plan actions.
- More intelligent than reactive agents.

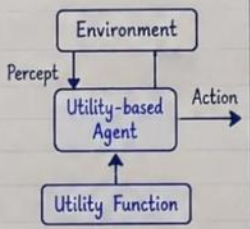
Example: Navigation system (e.g., Google Maps) that finds a path to a destination.



3) Utility-based Agents

- Choose actions that maximize a utility function.
- Handle trade-offs between multiple goals.
- Useful when there are multiple possible outcomes.

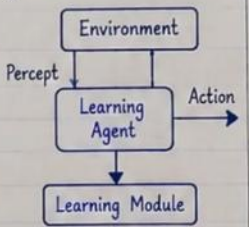
Example: Self-driving cars (balance safety, speed, comfort).



4) Learning Agents

- Improve performance over time from experience.
- Can learn from past interactions.
- Use techniques like reinforcement learning, supervised learning etc.

Example: Personalized assistants, recommendation systems.



4) Comparison of Agent Types

Feature	Reactive Agent	Goal-based Agent	Utility-based Agent	Learning Agent
Uses Past Experience	No	Partially	Partially	Yes
Uses Goals	No	Yes	Yes	Yes
Optimizes Outcomes	No	No	Yes	Yes
Can Learn	No	No	No	Yes
Complexity	Low	Medium	Medium	High
Example	Thermostat	GPS Navigation	Self-driving car	ChatGPT with memory

5) Key Takeaways

- Different types of agents are used based on the problem complexity.
- Reactive agents are simple, learning agents are most advanced.
- Agentic AI systems often combine multiple capabilities (e.g., goal-based + learning + tools).

★ Important Point:
Agentic AI is not limited to one type of agent. Real-world systems often use a combination of goal-based, utility-based and learning approaches.

Insta ID: @aditechnova18

SUBTOPIC: Agentic AI Architecture

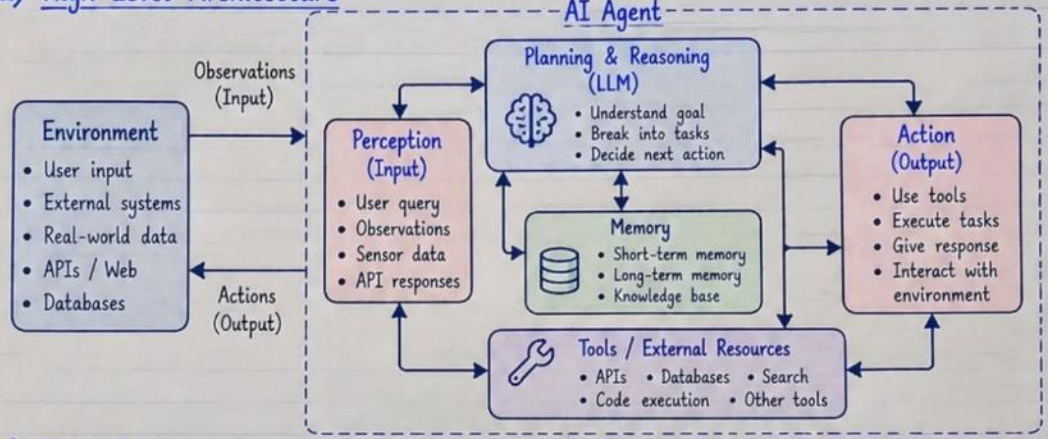
1) What is Agentic AI Architecture ?

Agentic AI architecture is the overall structure and design that enables an AI agent to perceive, reason, plan, use tools, remember past information and take actions in an environment to achieve specific goals autonomously.

Key Idea :

- Combines LLMs + Memory + Tools + Planning + Execution + Feedback loop
- Designed for autonomous goal achievement
- Can work with real-world systems and data
- More than just generating text

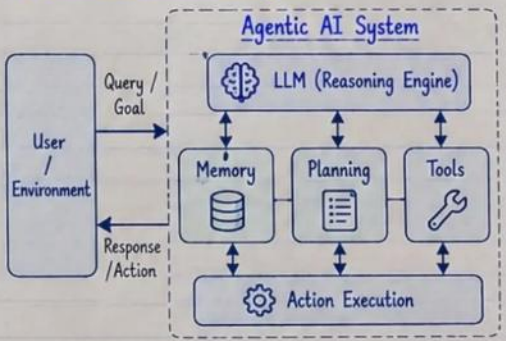
2) High Level Architecture



3) Core Components

1	Perception Module	• Takes input from user/environment • Processes and understands the input
2	Planning & Reasoning Module	• Uses LLM to analyse, plan and decide • Breaks complex goals into sub-tasks
3	Memory Module	• Stores past interactions, knowledge and context • Helps in maintaining long-term goals
4	Tools / Action Module	• Uses external tools, APIs and resources • Executes planned actions
5	Environment	• Real-world or digital environment • Provides feedback based on agent's actions

4) Detailed Block Diagram



5) Example Workflow

1. Receive Goal (User query) → 2. Understand & Plan (LLM) → 3. Retrieve Relevant Memory → 4. Use Tools (APIs / Search / Code) → 5. Take Action & Get Result → 6. Observe & Improve

6) Key Takeaways

- Agentic AI architecture integrates perception, reasoning, memory, tools and action.
- LLMs act as the core reasoning and planning engine.
- Tools and memory make agents capable of real-world task execution.
- The architecture is modular and can be extended based on use case.

★ Important Point :

- Different frameworks (LangChain, LangGraph, CrewAI etc.) implement variations of this architecture.
- The core components remain similar across most agentic AI systems.

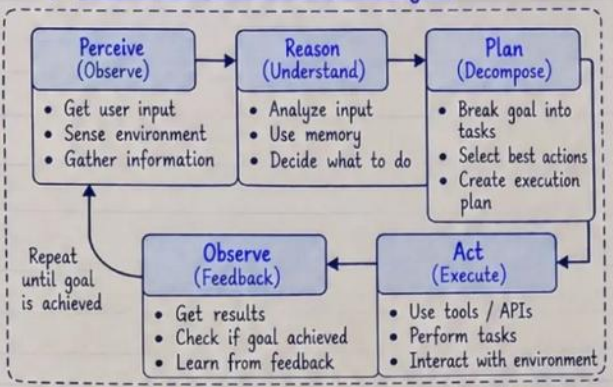
Insta ID: @ aditechnova18

SUBTOPIC: Agentic AI Workflow and Reasoning

1) What is Agentic AI Workflow ?

- It is the step-by-step process followed by an AI agent to achieve a goal.
- It involves perceiving the environment, reasoning about the information, planning actions, executing those actions and observing the results.
- The workflow runs in a continuous loop until the goal is achieved.

2) Overall Workflow of an AI Agent



3) Reasoning in Agentic AI

- Reasoning is the ability of the agent to think, analyze and decide the next best action based on current information, past experience and the goal.
- LLMs are used for reasoning using techniques like Chain-of-Thought, ReAct and self-reflection.

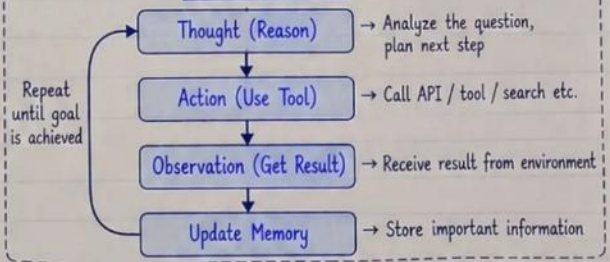
Key Reasoning Steps

1. Understand the current state (context)
2. Identify the goal
3. Retrieve relevant knowledge from memory
4. Analyze possible actions
5. Choose the best action
6. Evaluate the outcome and adapt

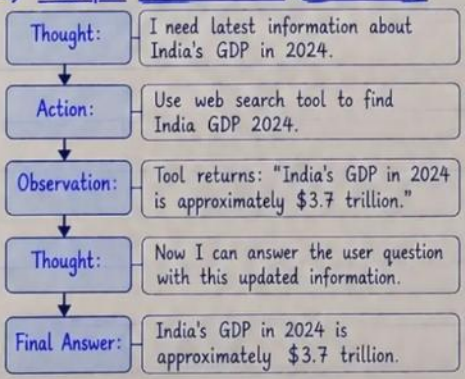
4) ReAct Pattern (Reason + Act)

- ReAct (Reasoning + Acting) is a popular approach where the agent alternates between reasoning and taking actions.
- It helps the agent use external tools and observe real-world feedback.
- Useful for tasks like search, coding, data analysis etc.

ReAct Workflow



5) Example: ReAct in Action (Search Task)



6) Other Reasoning Techniques

- Chain-of-Thought (CoT) – step by step reasoning
- Tree of Thoughts (ToT) – explore multiple reasoning paths
- Self-Consistency – generate multiple answers and choose best
- Self-Reflection – evaluate and improve own output
- Plan-and-Execute – plan the full task first, then execute
- ReAct – interleave reasoning and acting
- Reflexion – learn from past mistakes

7) Key Takeaways

- Agentic AI workflow is a continuous loop.
- Reasoning helps the agent make better decisions.
- ReAct is widely used for tool-using agents.
- Different reasoning techniques are useful for different tasks.
- The agent adapts based on feedback from the environment.

★ **Important Point:**
A good agent doesn't just generate an answer, it reasons, uses tools, takes actions and learns from feedback.

SUBTOPIC: LLMs and Prompt Engineering for Agents

1) Role of LLMs in Agentic AI

- LLMs act as the brain of an agent.
- They help in understanding user queries, reasoning, planning, tool use and generating responses.
- LLMs can break complex tasks into sub-tasks, decide which tools to use and analyze results.
- They work with memory, tools and environment to achieve goals autonomously.

2) LLM vs Traditional Models

Feature	Traditional ML Models	LLMs (for Agents)
Input Type	Structured data	Natural language
Output	Fixed / classification	Open-ended text
Knowledge	Limited, trained data	Large world knowledge
Reasoning	Rule-based	Advanced reasoning
Flexibility	Task-specific	General purpose
Use in Agent	Limited	Core component

3) Types of Prompts

1. System Prompt – Sets the role, behavior and rules.
2. User Prompt – Actual query or instruction.
3. Context Prompt – Provides background information.
4. Few-shot Prompt – Gives examples to guide output.
5. Chain-of-Thought Prompt – Encourages step-by-step reasoning.
6. Tool-use Prompt – Helps LLM decide when and how to use tools.
7. Output Format Prompt – Enforces structured output (JSON etc.).

4) Anatomy of a Good Prompt

- **Role** → Define the agent's role (e.g., You are a research assistant)
- **Context** → Provide necessary background (e.g., topic, data, goal)
- **Task** → Clearly state what to do (e.g., summarize, analyze, plan)
- **Constraints** → Specify limitations or rules (e.g., use only given data)
- **Output Format** → Ask for structured response (e.g., JSON, bullet points, table)

5) Prompting Techniques for Agents

- Zero-shot Prompting – Directly ask without examples.
- Few-shot Prompting – Provide examples in the prompt.
- Chain-of-Thought (CoT) – Ask for step-by-step reasoning.
- ReAct Prompting – Combine reasoning and action.
- Self-Consistency – Generate multiple answers and pick best.
- Reflection Prompting – Ask model to review and improve its own answer.
- Tool Calling Prompting – Instruct model to use specific tools/APIs.
- Structured Prompting – Ask for output in JSON or specific format.

6) Example Prompts

Simple Prompt

```
You are a helpful research assistant.
Summarize the given article in 5 bullet points.
Article: <text>
```

Prompt for Tool Use (Function Calling)

```
You are an AI agent with access to a web search tool.
When the user asks for latest information, use the
search_tool(query) function. After getting the result,
analyze it and provide a clear answer in bullet points.
```

7) Structured Output (JSON)

Agents often need structured output to work with tools.

Example Prompt:

```
Extract the name, date and location from the following text
and return the output in JSON format.
Text: "OpenAI launched GPT-5 on 10 July 2024 in USA."
```

Expected Output:

```
{
  "name": "GPT-5",
  "date": "10 July 2024",
  "location": "USA"
}
```

8) Best Practices

- Be clear and specific in instructions.
- Provide necessary context.
- Use structured output formats.
- Add examples for better accuracy.
- Set constraints to avoid unwanted results.
- Test and iterate prompts.
- Combine with memory and tools for better agent performance.

9) Key Takeaways

- LLMs are the core reasoning engine in agentic AI.
- Prompt engineering significantly improves agent performance.
- Different prompting techniques are useful for different scenarios.
- Structured outputs (e.g., JSON) help in tool integration and automation.

★ **Important Point:**
A well-designed prompt can make the agent more accurate, safer and capable of using tools effectively.

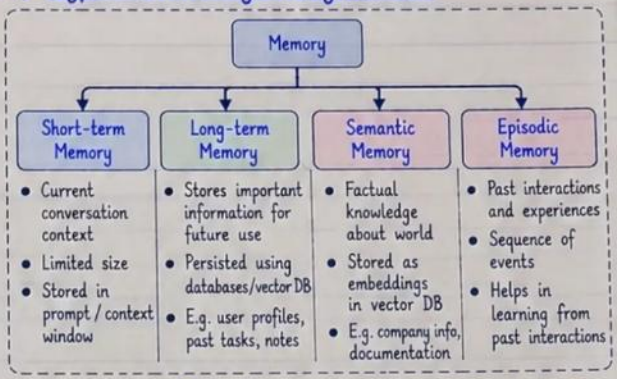
Insta ID: @aditechnova18

SUBTOPIC: Memory Systems in Agentic AI

1) Why Memory is Important ?

- LLMs are stateless – they don't remember past conversations by default.
- Memory helps an agent remember past interactions, user preferences, important facts and context.
- It improves continuity, personalization and decision making in long-term tasks.
- Without memory, the agent may repeat mistakes or give inconsistent responses.

2) Types of Memory in Agentic AI

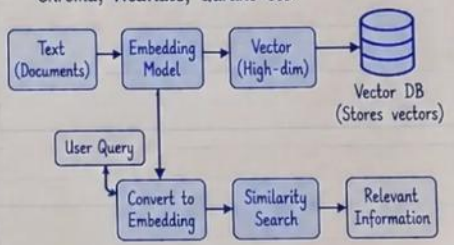


3) Short-term vs Long-term Memory

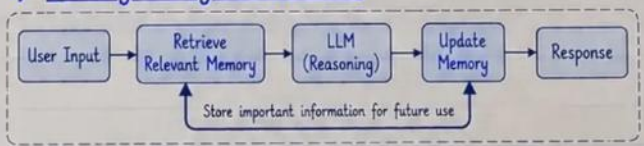
Feature	Short-term Memory	Long-term Memory
Purpose	Maintain current context	Store important info for future use
Duration	Temporary (current session)	Persistent (across sessions)
Storage	Prompt / context window	Database / Vector DB
Capacity	Limited (few thousand tokens)	Large (scalable)
Examples	Recent messages, current task	User preferences, past projects, important facts

4) Semantic Memory (Vector Database)

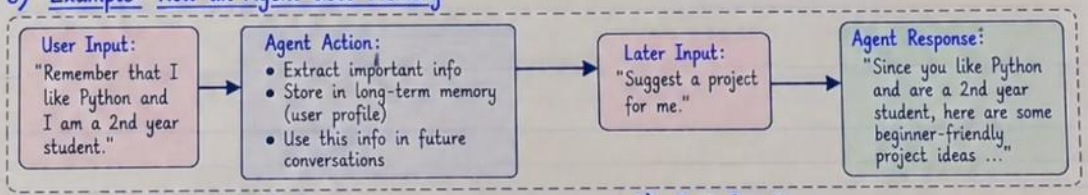
- Stores information as embeddings (vector representations) instead of plain text.
- Helps in finding relevant information using semantic search (meaning-based search).
- Common Vector Databases: Pinecone, FAISS, Chroma, Weaviate, Qdrant etc.



5) Memory in Agent Workflow



6) Example: How an Agent Uses Memory



7) Memory Storage Techniques

Technique	Description	Use Case
In-Memory	Store in prompt/context	Short-term conversation
Key-Value Store	Simple structured storage	User preferences, settings
Vector Database	Store embeddings	Semantic search, document retrieval
Relational Database	Structured data (SQL)	User profiles, logs, tasks
Graph Database	Store relationships	Knowledge graphs, complex relations
File Storage	Store text/documents	Notes, documents, history

8) Best Practices

- Store only important and relevant information.
- Use a combination of short-term + long-term memory.
- Use vector databases for unstructured data.
- Regularly update and clean old memories.
- Limit sensitive data and handle privacy carefully.
- Use summarization to compress long histories.
- Design memory retrieval to get only relevant context.

9) Key Takeaways

- Memory makes agents more useful and personalized.
- Different types of memory serve different purposes.
- Vector databases enable semantic memory.
- Good memory management improves accuracy, consistency and user experience.

Insta ID: @aditechnova18

SUBTOPIC: Tools, APIs and Function Calling

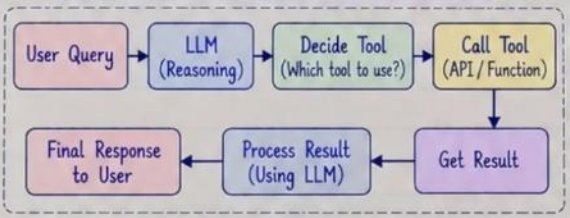
1) Why Tools are Important in Agentic AI ?

- LLMs are powerful but have limitations.
- They cannot directly access real-time data, databases or external systems.
- Tools and APIs allow agents to interact with the real world and perform useful actions.
- With tools, agents can search the web, fetch data, run code, query databases, send emails etc.
- This makes agents more capable, reliable and useful for real-world tasks.

2) Types of Tools

	Web Search Tools	Search the internet for latest information. (e.g., Google Search API, SerpAPI)
	Database Tools	Query structured data from databases. (e.g., MySQL, PostgreSQL)
	Code Execution Tools	Run code snippets (Python, etc.). (e.g., Python REPL, Jupyter)
	File & Document Tools	Read, write and analyze files. (e.g., PDF reader, File system)
	Communication Tools	Send emails, messages, notifications. (e.g., Gmail API, Slack API)
	Custom Tools / APIs	Domain-specific tools for special tasks. (e.g., weather API, booking API)

3) How Tool Use Works ? (General Flow)



4) Function Calling in LLMs

- Function calling allows the LLM to decide when to call a function/tool and with what parameters.
- The LLM does not execute the function itself, it only generates a structured request (usually in JSON).
- Our application/backend receives this request, executes the function and returns the result to the LLM.
- LLM then uses the result to generate a final answer.

5) Example: Function Calling (Get Weather)

User Query: "What is the weather in Kanpur today ?"

LLM Output (Function Call Request):

```
{
  "name": "get_weather",
  "arguments": {
    "location": "Kanpur"
  }
}
```

LLM decides to call the function with required parameters.

Function Execution (Backend):

```
result = get_weather(location="Kanpur")
```

Final Response (by LLM using result):

"The current weather in Kanpur is 28°C with clear skies."

6) Common APIs Used with Agents

Purpose	API / Tool	Description
Web Search	Google Search API (SerpAPI)	Search real-time information
Weather	OpenWeather API	Get weather data
Maps	Google Maps API	Location, routes, places
Database	MySQL / PostgreSQL APIs	Query structured data
File Handling	Python file tools	Read/write documents
Email	Gmail API	Send and read emails
Custom	Any REST API	Domain specific tasks

8) Defining a Function (OpenAI Style)

```
{
  "name": "get_weather",
  "description": "Get current weather for a location",
  "parameters": {
    "type": "object",
    "properties": {
      "location": {
        "type": "string",
        "description": "City name"
      }
    },
    "required": ["location"]
  }
}
```

Key Fields:

- name → function name
- description → what it does
- parameters → input schema
- required → list of mandatory params

7) Best Practices for Tool Use

- Use clear and well-defined function descriptions.
- Provide correct parameter schema (type, description, required).
- Handle errors and invalid inputs gracefully.
- Validate tool outputs before using them.
- Combine multiple tools for complex tasks.
- Keep tools secure and avoid sensitive data exposure.

9) Key Takeaways

- Tools and APIs extend the capabilities of LLMs.
- Function calling enables structured and reliable tool use.
- Agents can use multiple tools to solve complex real-world tasks.
- Proper tool design and error handling improve performance and safety.

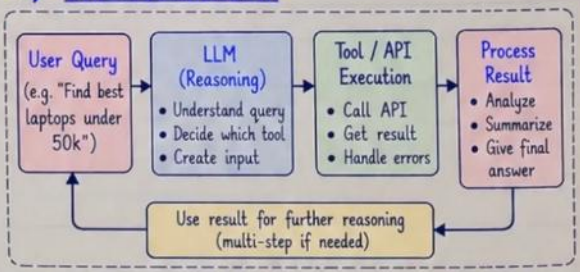
Insta ID: @ aditechnova18

SUBTOPIC: Tool Use Strategies, APIs and Real-world Integration

1) Wol Use Strategies in Agents

- An agent decides when, which and how to use tools to solve a task.
- It may use multiple tools in a sequence.
- The agent analyzes the user query, chooses the right tool, prepares the input, executes it and uses the result to continue reasoning.
- Good tool use makes the agent more accurate, capable and useful in real-world applications.

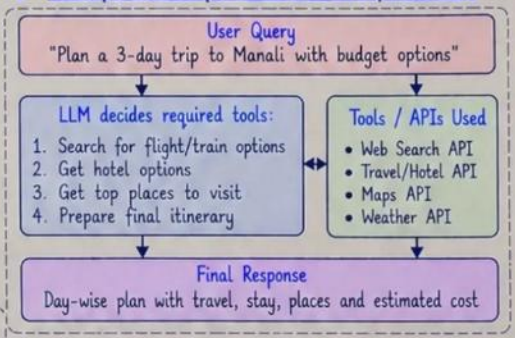
2) Tool Use Workflow



3) Tool Selection Strategies

1. Rule-based Selection – use predefined rules to choose tools.
2. LLM-based Selection – let the LLM decide based on tool descriptions.
3. ReAct Style – reason about which tool to use and why.
4. Plan-then-Execute – create a plan with multiple tool calls.
5. Self-Ask / Decompose – break complex tasks into smaller subtasks and choose tools for each.

4) Example: Multiple Tool Use (Trip Planner)



5) Example: Web Search Tool (SerpAPI)

Function Definition (Schema)

```
{
  "name": "search_web",
  "description": "Search the internet
    for latest information.",
  "parameters": {
    "tupe": "object",
    "properties": {
      "query": { "tupe": "string",
        "description": "Search query" },
      "num_results": { "tupe": "integer"
      }
    },
  } "required": ["query"]
}
```

Example Function Call

```
{
  "name": "search_web",
  "arguments": {
    "query": "best laptops
      under 50000 in 2024",
  } "num_results": 5
}
```

Possible API Response (Simplified)

```
{
  "results": [
    {"title": "Best Laptops 2024",
      "link": "https://....",
      "snippet": "Top laptops under
                  50k ..."},
    ...
  ]
}
```

6) Handling Errors and Uncertainty

- Check if the tool is available and working.
- Handle API errors (network issues, invalid input, rate limit).
- If a tool fails, the agent can:
  - Retry with modified input.
  - Use an alternative tool.
  - Ask the user for clarification.
  - Continue with partial information.
- Always validate and verify important results.
- Inform the user when data may be uncertain.

7) Real-world Examples of Tools in Agents

Use Case	Tool / API	Purpose
Web Search	Google Search API, SerpAPI	Get latest information
Weather	OpenWeather API	Get current weather data
Maps / Location	Google Maps API	Get places, routes, travel time
News	News API	Get latest news
Finance	Stock/Finance API	Get stock prices, market data
Code Execution	Python REPL, Jupyter	Run code and analyze results
File Handling	File system tools	Read, write and process files
Email	Gmail API	Send and read emails
Custom	Any REST API	Domain specific tasks (e.g. booking)

8) Best Practices

- Provide clear tool descriptions to the LLM.
- Use well-defined input schemas.
- Validate inputs before calling tools.
- Handle errors and edge cases.
- Combine multiple tools for complex tasks.
- Keep API keys secure (use environment variables).
- Limit tool usage to prevent abuse and high costs.
- Prefer reliable and well-documented APIs.
- Test the agent with different queries.

9) Key Takeaways

- Tools and APIs bridge the gap between LLMs and the real world.
- Proper tool use makes agents more capable and practical.
- Good design, error handling and validation are essential.
- Multiple tools can be combined to solve complex real-world tasks.

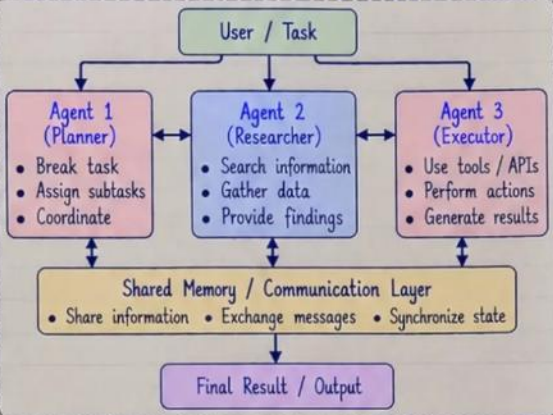
Insta ID: @ aditechnova18

SUBTOPIC: Multi-Agent Systems (Agents Working Together)

1) What are Multi-Agent Systems?

- A Multi-Agent System (MAS) consists of multiple autonomous agents that work together (or sometimes compete) to achieve a goal.
- Each agent can perceive, reason, plan and act independently.
- Agents communicate with each other and may share information, tools or resources.
- Useful for complex tasks that require multiple skills, perspectives or parallel processing.

2) Architecture of a Multi-Agent System



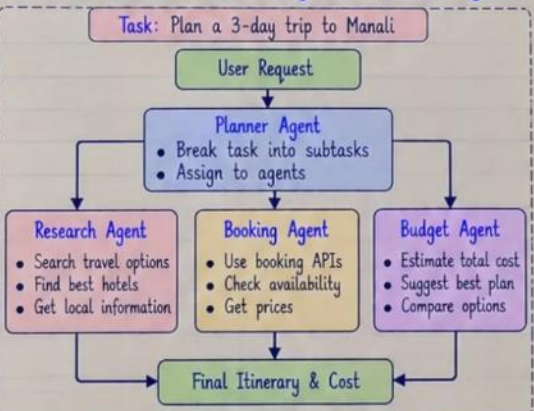
3) Types of Multi-Agent Collaboration

1. Cooperative Agents – work together to achieve a common goal. (e.g., research team)
2. Competitive Agents – compete for resources or rewards. (e.g., trading bots)
3. Hybrid (Coop + Compete) – some cooperate and some compete. (e.g., multi-player games)
4. Hierarchical – agents have roles and follow a leader. (e.g., supervisor and worker agents)
5. Peer-to-Peer – all agents are equal and communicate directly. (e.g., decentralized systems)

4) Communication Between Agents

- Agents communicate by sharing messages, observations, plans or results.
- Common communication methods:
  1. Direct Messaging – agent sends message to another.
  2. Shared Memory – common storage accessible by all agents.
  3. Message Passing – structured messages (e.g., JSON).
  4. Blackboard Model – agents read/write to a shared board.
  5. Event-driven – agents react to events in the environment.

5) Example: Travel Planning with Multiple Agents



6) Real-world Applications

- Travel and logistics planning
- E-commerce (product search, price comparison)
- Healthcare (multiple agents for diagnosis, research, care)
- Autonomous vehicles (multi-agent coordination)
- Robotics (multiple robots working together)
- Finance (trading agents)
- Software development (one agent for coding, one for testing)
- Customer support (multiple specialized agents)

7) Advantages and Challenges

Advantages	Challenges
• Handle complex tasks	• Coordination between agents
• Parallel processing (faster)	• Communication overhead
• Specialization (each agent has specific skills)	• Conflict resolution
• More accurate and reliable results	• Consistency of information
• Scalable to large problems	• Higher computational cost
	• Security and trust issues

8) Example: Agents in Software Development

Agent	Role	Tasks
Planner Agent	Project planning	Break requirements, assign tasks
Coder Agent	Write code	Generate code using IDE/tools
Tester Agent	Test code	Run tests, find bugs
Reviewer Agent	Code review	Improve code quality
Deployer Agent	Deployment	Deploy to server/cloud
Monitor Agent	Monitoring	Track performance, alert issues

9) Key Takeaways

- Multiple agents can work together to solve complex real-world tasks.
- Each agent can have a specific role and set of tools.
- Good communication and coordination are essential.
- MAS is useful in many domains like travel, healthcare, finance and robotics.
- It improves efficiency, scalability and reliability.

Insta ID: @ aditechnova18

SUBTOPIC: Planning, Reasoning and Decision Making in Agentic AI

1) What is Planning in Agentic AI?

- Planning is the process of breaking a complex goal into a sequence of smaller steps (actions) to achieve the goal.
- An agent uses the LLM's reasoning ability and planning techniques to decide what to do next.
- It helps handle multi-step tasks and ensures better, consistent and goal-oriented behavior.

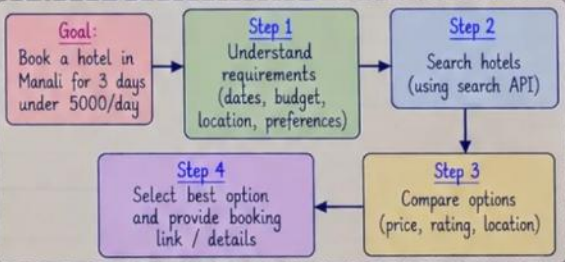
2) Types of Planning

Type	Description	Example
Reactive Planning	Decide next action based on current state (no long-term plan)	Simple Q&A, real-time response
Deliberative Planning	Plan multiple steps ahead using reasoning	Book a trip, research, compare and finalize
Hierarchical Planning	Break goal into high-level and low-level subgoals	Plan → Book → Pay → Generate itinerary
Goal-based Planning	Focus on achieving a specific goal	"Find best laptop under 50k"
Utility-based Planning	Choose actions that maximizes expected utility	Compare multiple options and pick best

3) Planning Techniques

1. Chain-of-Thought (CoT) – Step-by-step reasoning.
2. Tree of Thoughts (ToT) – Explore multiple possible paths (branches).
3. ReAct (Reason + Act) – Combine reasoning and action using tools.
4. Plan-and-Execute – First create a plan, then execute it step by step.
5. Self-Ask – Break complex questions into smaller sub-questions.
6. Reflection / Self-Critique – Review and improve the plan based on results.
7. Decomposition – Break large tasks into smaller subtasks.

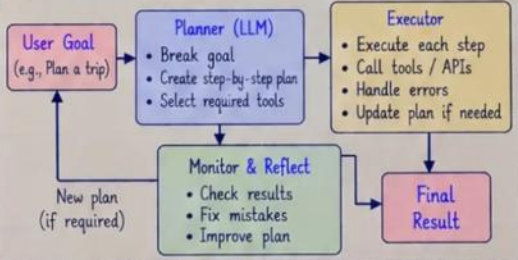
4) Example: Task Planning (Book a Hotel)



5) Reasoning Approaches

Approach	Idea	When to Use
Zero-shot	Directly reason without examples	Simple tasks
Few-shot	Use few examples in prompt	Moderate tasks
Chain-of-Thought	Step-by-step reasoning	Complex reasoning
Tree-of-Thoughts	Explore multiple reasoning paths	Open-ended or multi-path problems
ReAct	Reason and use tools/actions	Real-world tasks
Self-Consistency	Generate multiple solutions and pick best	Better accuracy

6) Plan-and-Execute Workflow



7) Example: Step-by-Step Plan Generation

User Query:

"I want to learn Java in 1 month. Create a study plan with topics and daily schedule."

LLM Generated Plan (Simplified):

1. Understand basics (Day 1-3)
2. OOP concepts (Day 4-8)
3. Collections and File Handling (Day 9-12)
4. Exception Handling (Day 13-15)
5. Mini Projects (Day 16-25)
6. Revision and Practice (Day 26-30)

8) Challenges in Planning and Reasoning

- LLMs may generate incorrect or illogical plans.
- Handling uncertainty and incomplete information.
- Long-term planning can be difficult due to context limits.
- Balancing exploration (trying new options) vs exploitation (using known good options).
- Ensuring the plan is feasible with available tools and resources.

9) Key Takeaways

- Planning helps agents break complex goals into manageable steps.
- Different reasoning techniques improve decision making.
- Combining planning with tools enables real-world problem solving.
- Reflection and self-critique improve plan quality.
- A good plan leads to more accurate, reliable and useful results.

Insta ID: @aditechnova18

SUBTOPIC: Agentic AI Frameworks (LangChain, AutoGen, CrewAI, etc.)

1) What are Agentic AI Frameworks?

- Agentic AI frameworks provide ready-made libraries, tools and abstractions to build, run and manage AI agents.
- They simplify complex tasks like tool use, memory, planning, multi-agent communication and deployment.
- Developers can build real-world agent systems faster using these frameworks.

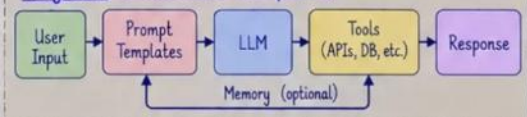
2) Popular Agentic AI Frameworks

Framework	Created By	Language	Main Focus	Use Cases
LangChain	LangChain Inc.	Python (also JS)	LLM apps, agents, tools	Agent with tools, RAG, workflows
AutoGen	Microsoft	Python	Multi-agent conversation	Collaborative agents, research
CrewAI	CrewAI	Python	Role-based multi-agent systems	Team of agents for complex tasks
Semantic Kernel	Microsoft	Python, C#, Java	Integrating LLMs with enterprise tools	Business apps, enterprise agents
LlamaIndex	LlamaIndex	Python	Data indexing and retrieval	RAG, knowledge agents

3) LangChain Framework

- One of the most popular frameworks for building LLM applications and agents.
- Provides modules for prompts, chains, tools, agents, memory and vector databases.
- Supports integration with many LLMs and external APIs.

LangChain Architecture (Simplified)



4) Key Components of LangChain

Component	Purpose
Prompt Templates	Create dynamic prompts
Chains	Combine multiple steps
Agents	Use tools to perform actions
Tools	Connect to external APIs, databases, etc.
Memory	Remember past interactions
Vector Stores	Store and retrieve embeddings
Callbacks	Monitor and log agent execution

5) Example: Simple Agent with a Tool (LangChain)

```
from langchain.agents import initialize_agent, Tool
from langchain.llms import OpenAI
from langchain.tools import SerpAPIWrapper

# Define a tool
search = SerpAPIWrapper()

# Initialize LLM
llm = OpenAI(temperature=0)

# Create tool list
tools = [Tool(
    name="Search",
    func=search.run,
    description="Search the internet for current information"
)]

# Initialize agent
agent = initialize_agent(tools, llm, agent="zero-shot-react-description")

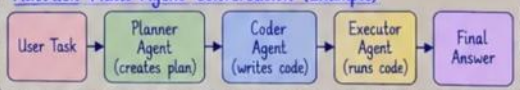
# Run agent
response = agent.run("What is the latest news about ISRO?")
print(response)
```

This agent can search the internet using SerpAPI and answer real-time questions.

6) AcewAI Framework

- Developed by Microsoft for building multi-agent conversational systems.
- Agents can talk to each other, plan, assign tasks and solve problems collaboratively.
- Useful for research automation, code generation, data analysis and more.

AutoGen Multi-Agent Conversation (Example)



7) CrewAI Framework

- Focuses on role-based multi-agent systems.
- You can define agents with specific roles, goals and tools.
- Agents work together like a team to complete complex tasks.
- Easy to use and good for real-world applications.

CrewAI Team Structure (Example)



8) Comparison of Frameworks

Feature	LangChain	AutoGen	CrewAI	Semantic Kernel
Ease of Use	Easy	Moderate	Easy	Moderate
Multi-Agent Support	Yes (basic)	Yes (strong)	Yes (strong)	Yes
Tool Integration	Excellent	Good	Good	Good
Memory Support	Yes	Yes	Yes	Yes
Best For	General agents	Collaborative agents	Role-based agents	Enterprise apps

9) Key Takeaways

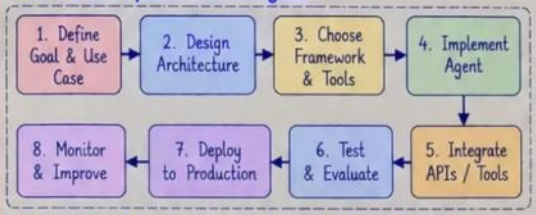
- Agentic AI frameworks make it easier to build powerful agents.
- LangChain is great for single agents with tool use.
- AutoGen is best for multi-agent collaboration.
- CrewAI focuses on team-based agents with roles.
- Choose the framework based on your use case, complexity and required features.

SUBTOPIC: Building and Deploying Agentic AI Systems (End-to-End)

1) End-to-End Development Process

- Building an Agentic AI system involves multiple steps from idea to deployment.
- It includes requirement analysis, designing agent architecture, selecting tools, implementation, testing and deployment.
- Each step ensures the agent is goal-oriented, reliable and useful in real-world applications.

2) Development Lifecycle



3) Step 1: Define Goal and Requirements

- Clearly define the problem the agent will solve.
- Identify target users, expected inputs, outputs and constraints.
- Decide whether single-agent or multi-agent system is needed.

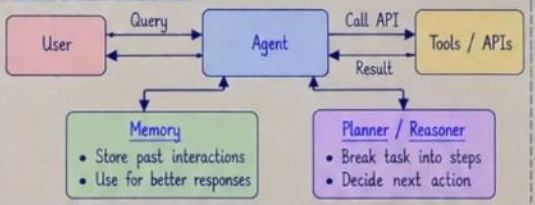
Example Requirement (Travel Planner Agent)

- Goal: Plan a 3-day trip within budget.
- Inputs: destination, budget, travel dates, preferences.
- Outputs: detailed itinerary with cost, hotels, travel options.
- Constraints: budget limit, real-time availability.

4) Step 2: Design Agent Architecture

- Decide the agent type (single/multi-agent).
- Choose reasoning and planning techniques.
- Decide which tools, APIs and memory will be used.
- Design the workflow and communication between components.

Example Architecture



5) Step 3: Select Framework and Tools

Component	Popular Options
Frameworks	LangChain, AutoGen, CrewAI, Semantic Kernel, LlamaIndex
LLM	OpenAI GPT, Anthropic Claude, Google Gemini, Llama
Tools / APIs	Web Search API, Database, Custom APIs (REST)
Vector Database	Pinecone, ChromaDB, FAISS, Weaviate
Deployment	Docker, Streamlit, FastAPI, Cloud platforms (AWS, GCP)

6) Step 4: Implement the Agent

- Write the code using the selected framework.
- Integrate required tools and APIs.
- Implement memory, planning and reasoning.
- Handle errors and edge cases.
- Keep the design modular and scalable.

Example: LangChain Agent (Simple)

```
from langchain.agents import initialize_agent, Tool
from langchain.llms import OpenAI
from langchain.tools import SerpAPIWrapper

llm = OpenAI(temperature=0)
search = SerpAPIWrapper()

tools = [ Tool(
        name = "Search",
        func = search.run,
        description = "Search the internet for information"
    )]

agent = initialize_agent(tools, llm, agent="zero-shot-react-description")
response = agent.run("Plan a 3-day trip to Manali under 5000")
print(response)
```

This code creates
- a simple agent
- uses web search tool
- takes user query
- gives real-time answer

7) Step 5: Testing and Evaluation

- Test the agent with different types of inputs.
- Check correctness, relevance and reliability of responses.
- Evaluate how well it handles errors, missing data and complex queries.
- Compare outputs with expected results.

Evaluation Metric	Description
Accuracy	Correctness of the response
Relevance	How relevant the answer is to the query
Tool Usage	Whether the right tools are called
Latency	Time taken to generate response
Robustness	Handles errors and edge cases
User Satisfaction	Overall usefulness of the response

8) Step 6: Deployment

- Deploy the agent using platforms like Streamlit, FastAPI, Docker or cloud services (AWS, GCP, Azure).
- Make the agent accessible via web app, API or chatbot interface.
- Ensure proper security, authentication and rate limits.

Deployment Options

Local Deployment	Container Deployment	Cloud Deployment
• Streamlit • FastAPI (localhost)	• Docker • Docker Compose	• AWS, GCP, Azure • Serverless (Lambda)

9) Monitoring and Improvement

- Monitor agent usage, performance and errors.
- Collect user feedback and improve prompts, tools and logic.
- Update memory and knowledge base regularly.
- Add new tools or capabilities based on user needs.

10) Key Takeaways

- Building an Agentic AI system involves planning, designing, implementation, testing, deployment and continuous improvement.
- Use the right framework and tools for your use case.
- Proper testing and monitoring make the agent reliable and scalable.
- Iterate and improve based on real-world usage and feedback.

Insta ID: @ aditechnova18

SUBTOPIC: Real-World Applications, Safety, Ethics and Challenges

1) Real-World Applications of Agentic AI

- Agentic AI is being used in many industries to automate complex tasks and make better decisions.
- It combines LLMs, tools, memory and planning to work independently for real-world goals.

Domain	Use Cases	Example
Travel & Hospitality	Trip planning, booking, itinerary management	AI travel agent (book flights, hotels)
Healthcare	Patient support, report analysis, appointment booking	Symptom checker agent, research agent
Finance	Market analysis, portfolio management	Investment advisor agent
Education	Personalized learning, research assistance	Study planner agent
E-commerce	Product search, comparison, recommendations	Shopping assistant agent
Software Development	Code generation, testing, debugging, documentation	AI coding agent
Research & Analytics	Literature review, data analysis, report generation	Research agent with web + tools
Customer Support	Automated support, ticket handling	Multi-agent support system

2) Case Study: Autonomous Research Agent

Goal:
Research a topic, collect latest papers, summarize them and create a report.

Workflow

User Query
↓
Plan (break into steps)
↓
Search Tools (web, arXiv, APIs)
↓
Read & Summarize
↓
Generate Final Report

Uses LLM + Tools

Memory for past results

3) Safety and Reliability

- Agentic AI systems can take powerful actions, so safety is very important.
- We need to make sure the agent is aligned, controllable and does not harm users or systems.

Safety Aspect	Description
Alignment	Ensure agent follows human intent and values.
Tool Safety	Restrict dangerous tools and actions.
Input Validation	Check and sanitize user inputs.
Output Control	Filter harmful or incorrect responses.
Human-in-the-Loop	Keep humans in control for critical tasks.
Monitoring	Track agent actions, logs and performance.
Failsafes	Stop or rollback in case of unexpected behavior.

4) Ethical Considerations

- Agentic AI can impact privacy, fairness and society.
- We must use it responsibly and ethically.

Ethical Issue	Description
Privacy	Protect user data and personal information.
Bias	Avoid biased outputs and unfair decisions.
Transparency	Make agent actions explainable.
Accountability	Clearly define who is responsible.
Misinformation	Prevent spreading false or misleading information.
Job Impact	Consider effects on employment and society.
Fair Access	Ensure benefits are available to everyone.

5) Challenges in Real-World Deployment

- Building and using agentic AI in real applications is not easy.
- There are technical, safety, cost and social challenges.

Challenge	Explanation
Reliability	Agents may make mistakes or incorrect decisions.
Hallucinations	LLMs can generate false or unrealistic information.
Tool Failures	APIs or tools may be unavailable or return errors.
Long-term Planning	Difficult to handle very complex, multi-step tasks.
Memory Management	Storing and retrieving relevant information is challenging.
Cost	Using multiple tools and long context can be expensive.
Latency	Real-time responses can be slow due to multiple steps.
Security Risks	Agents may be vulnerable to prompt injection or misuse.
Scalability	Managing many agents in production can be complex.

6) Best Practices for Safe and Effective Use

1. Clearly define the goal and scope of the agent.
2. Use reliable frameworks and well-tested tools.
3. Implement proper error handling and fallback mechanisms.
5. Monitor and log agent actions.
6. Use human-in-the-loop for critical decisions.
7. Validate and verify important outputs.
8. Follow ethical guidelines and legal regulations.

7) Future Trends and Opportunities

- More advanced multi-agent systems working together.
- Better memory, reasoning and planning capabilities.
- Integration with real-world devices (IoT, robotics).
- Domain-specific agents for healthcare, education, finance, etc.
- Safer and more reliable agent frameworks.
- Wider adoption in industry and everyday life.
- Potential to solve complex global challenges.

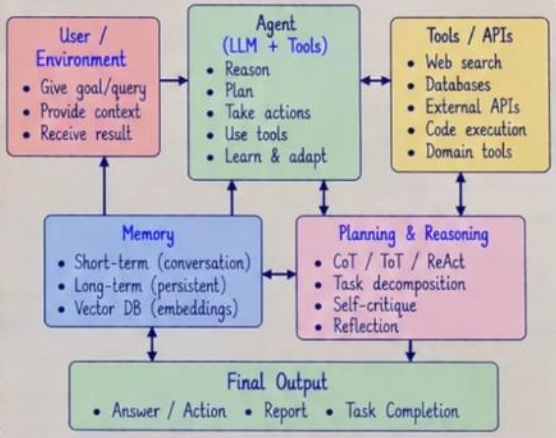
8) Key Takeaways

- Agentic AI has many real-world applications and huge potential.
- Safety, ethics and reliability are critical for responsible use.
- There are still many challenges to solve.
- Following best practices can help build useful and trustworthy agents.

Insta ID: @aditechnova18

SUBTOPIC: Summary, Interview Questions and Final Revision

1) Complete Agentic AI System Overview



2) Comparison: Normal AI vs Agentic AI

Feature	Normal AI (Chatbot)	Agentic AI
Goal Handling	Single-turn	Multi-step, complex goals
Tool Use	Limited / None	Uses multiple tools & APIs
Autonomy	Low	High
Memory	Usually short-term	Short-term + Long-term
Planning	Minimal	Advanced planning & reasoning
Real-world Actions	No	Yes (via tools/APIs)
Example	Answer a question	Plan a trip, book hotels, create report, analysis, etc.

3) Common Tools and Libraries

Category	Tools / Libraries	Purpose
Frameworks	LangChain, AutoGen, CrewAI, Semantic Kernel	Build and manage agents
LLMs	OpenAI GPT, Claude, Gemini, Llama	Reasoning and generation
Vector DB	Pinecone, Chroma, Weaviate	Store and retrieve embeddings
Web Search	SerpAPI, Tavily, Bing API	Real-time information
Code Execution	Python REPL, Docker	Run code and analysis
APIs Integration	REST APIs, OpenAPI	Connect to external services
Deployment	Streamlit, FastAPI, Docker, AWS, Azure	Deploy agent applications
Monitoring	LangSmith, Weights & Biases	Track, evaluate and debug

4) Important Terminologies (Quick Revision)

- **Agent**: AI system that can perceive, reason and act.
- **Tool**: External function or API used by agent.
- **Memory**: Stores past interactions and context.
- **Planning**: Breaking goal into smaller steps.
- **Reasoning**: Deciding next action using logic.
- **Multi-Agent System**: Multiple agents working together.
- **Vector Database**: Stores embeddings for semantic search.
- **RAG**: Retrieval-Augmented Generation (use external knowledge).
- **Prompt Template**: Structured input for LLM.
- **Chain**: Sequence of calls and operations.
- **Hallucination**: Generating false information.
- **ReAct**: Reason + Act approach.
- **CoT**: Chain-of-Thought.
- **ToT**: Tree-of-Thoughts.
- **Embedding**: Vector representation of text.

5) Interview Questions (with Short Answers)

1	What is Agentic AI ?	AI system that can plan, use tools, take actions and achieve goals autonomously.
2	How is it different from normal AI ?	Normal AI answers queries, while Agentic AI can perform multi-step tasks using tools and planning.
3	What are common frameworks ?	LangChain, AutoGen, CrewAI, Semantic Kernel.
4	What is RAG ?	Retrieval-Augmented Generation. It uses external knowledge (e.g., vector DB) to give better answers.
5	What is the role of memory ?	Stores past interactions and relevant information for better decision making.
6	What are planning techniques ?	CoT, ToT, ReAct, Plan-and-Execute, Self-Ask, Reflection.
7	How do agents use tools ?	Shey call external APIs, databases, web search or code execution tools to perform actions.
8	What are challenges in Agentic AI ?	Reliability, hallucinations, tool failures, high cost, latency, security risks.
9	How can we make agents safe ?	Input validation, output filtering, human-in-the-loop, monitoring and following ethical guidelines.
10	What are real-world applications ?	Travel planning, healthcare, finance, Customer support, education, automation, research, e-commerce, etc.

6) Final Revision Checklist

Topic	✓	Topic	✓
Basics of Agentic AI	☐	Example Applications	☐
Types of Agents	☐	Safety and Reliability	☐
Architecture and Components	☐	Ethical Considerations	☐
Planning Techniques	☐	Challenges and Limitations	☐
Reasoning Approaches	☐	Best Practices	☐
Memory in Agents	☐	Future Trends	☐
Tools, APIs and Function Calling	☐	Important Terminologies	☐
Popular Frameworks	☐	Comparison (Normal vs Agentic)	☐
Multi-Agent Systems	☐	Interview Questions	☐
Development Lifecycle	☐	Key Takeaways	☐

7) Key Takeaways

- Agentic AI combines LLMs, tools, memory and planning to achieve complex goals.
- It can work autonomously and handle multi-step real-world tasks.
- Frameworks like LangChain, AutoGen and CrewAI make development easier.
- Planning, reasoning and memory are essential for reliable agents.
- Safety, ethics and proper evaluation are very important.
- Agentic AI has huge potential in many industries and will continue to grow.
- It is an exciting field with many real-world opportunities.

8) Thank You !

Thanks for going through these notes!
Keep Learning, Keep Exploring and Keep Building 🚀
Follow @aditechnova18 for more such handwritten notes.

Stay Curious ☺
Stay Consistent
You Can Do It !
— Aditya Kumar
@aditechnova18

Agentic AI — Thank You Page

Insta ID: @aditechnova18

Thank You !

Thank you so much for going through these **Agentic AI notes !**

I really hope these notes helped you to understand the concepts clearly and in a simple way.

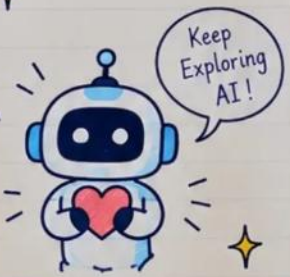
I have tried to cover everything from basics to advanced topics with examples, diagrams and key takeaways so that you can use this as a complete revision resource.

Keep Learning
Keep Exploring
Keep Growing
You Can Do It !

What You Learned ?

- [x] Introduction and core concepts of Agentic AI
- [x] Architecture, tools and frameworks
- [x] Planning, reasoning and decision making
- [x] Building and deploying agentic systems
- [x] Reat-world applications, safety, ethics and challenges

A complete 15-page revision to understand Agentic AI end-to-end !



Next Steps for You

1. Revise these notes regularly.
2. Practice with real-world projects.
3. Explore latest tools and frameworks.
4. Keep solving problems and building small agents.
5. Stay updated with new research and trends.

Consistency is the key to master any topic !

Remember !

- Take one step at a time.
- Don't just read, try and apply.
- Make your own notes.
- Build projects and experiment.
- It's okay to make mistakes.
- Learning is a continuous journey.
- Stay curious and keep exploring new possibilities !

Follow for More !

@aditechnova18 → For more handwritten notes, study resources and tech content.

Subscribe → For tutorials, projects and more.

Save these notes → So you can easily revise later.

Share with your friends → It might help them too !

Let's Learn Build and Grow Together !



"Knowledge becomes more powerful when it is shared."

— Aditya Kumar
@aditechnova18